

Процесс построения изображения

Быковских Дмитрий Александрович

19.09.2026

- Структура 3D-сцены
- Графический стек приложения
- Распределение вычислений
- Модель графического конвейера
- Характеристики видеокарты

- Структура 3D-сцены
- Графический стек приложения
- Распределение вычислений
- Модель графического конвейера
- Характеристики видеокарты

Из скольких объектов состоит сцена?

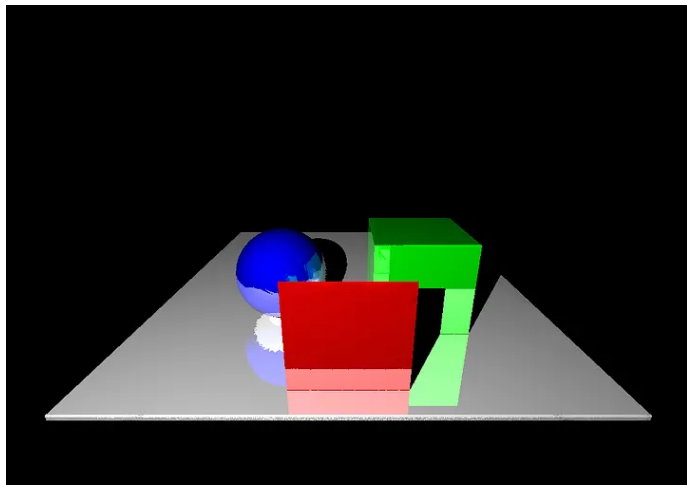


Рис. 1: Простая 3D-сцена

2026-09-19

Построение изображений

└ Процесс построения изображения

└ Из скольких объектов состоит сцена?

Из скольких объектов состоит сцена?

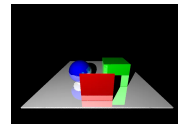


Рис. 1: Простая 3D-сцена

Рендеринг сцены

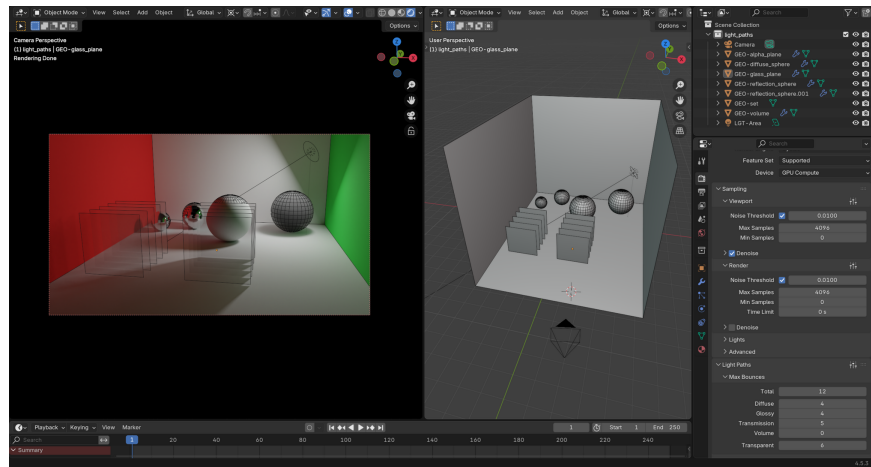


Рис. 2: Рендеринг сцены в blender

2026-09-19

Построение изображений

└─Процесс построения изображения

└─Рендеринг сцены

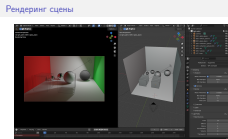
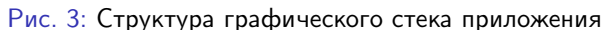


Рис. 2: Рендеринг сцены в blender

Структура 3D-сцены:

- Объекты -> Полигоны -> Вершины -> Атрибуты
- Источники света -> Атрибуты
- Камера -> Атрибуты



2026-09-19

Построение изображений

Графический стек приложения

└ Многоуровневая модель графического стека приложения



Терминология

Драйвер (видеодрайвер) — системный модуль, управляющий конкретным GPU и реализующий графический API в данной ОС.

Спецификация — формальное описание графического API: функции, типы, состояния и гарантии поведения реализации (OpenGL, Vulkan). Это документ, а не исполняемый код.

Графический API — программный интерфейс, через который приложение отдаёт работу GPU (контекст, буферы, текстуры, шейдеры, вызовы отрисовки).

Графическая библиотека — реализация спецификации (Mesa) либо тонкая обёртка над API: загрузка функций, создание контекста, математика, загрузка изображений и моделей.

Графический фреймворк — каркас приложения: задаёт архитектуру и цикл работы, вызывает код пользователя (инверсия управления). Выше библиотеки, без готовых игровых подсистем (физика, ИИ, сеть).

Графический движок — подсистема визуализации: сцена, материалы, освещение, отправка команд в API.

Игровой движок — надстройка над API/библиотекой: графический движок плюс физика, анимация, аудио, I/O, скрипты; часто сеть.

Графические редакторы — программы для создания и правки ассетов (растр, вектор, 3D, воксели).

API (Application Programming Interface) и графические библиотеки

OpenGL (1992, Silicon Graphics Inc.)

Открытая кроссплатформенная спецификация для работы с 2D- и 3D-графикой

Mesa (1995, Brian Paul)

Свободная реализация графических API: OpenGL (позже Vulkan и др.) с открытым исходным кодом

DirectX (1995, Microsoft)

Пакет API для разработки игр и мультимедийных приложений на платформе Windows

WebGL (2011, Khronos Group)

Графический API для веб-браузеров

Mantle (2013, AMD)

Спецификация низкоуровневого API

Vulkan (2016, Khronos Group)

Низкоуровневый (требует явного управления памятью и ресурсами), высокопроизводительный кроссплатформенный графический API

2026-09-19

Построение изображений

└ Графический стек приложения

└ API (Application Programming Interface) и графические библиотеки

API (Application Programming Interface) и графические библиотеки

OpenGL (1992, Silicon Graphics Inc.)
Открытая кроссплатформенная спецификация для работы с 2D- и 3D-графикой
Mesa (1995, Brian Paul)
Свободная реализация графических API: OpenGL (позже Vulkan и др.) с открытым исходным кодом
DirectX (1995, Microsoft)
Пакет API для разработки игр и мультимедийных приложений на платформе Windows
WebGL (2011, Khronos Group)
Графический API для веб-браузеров
Mantle (2013, AMD)
Спецификация низкоуровневого API
Vulkan (2016, Khronos Group)
Низкоуровневый (требует явного управления памятью и ресурсами), высокопроизводительный кроссплатформенный графический API

Пример взаимодействия спецификации, API и библиотек

1. Спецификация OpenGL 4.5 описывает контракт:
`DrawArrays(TRIANGLE_FAN, first, count)` строит примитивы из VAO.
2. API — вызов в коде:
`glDrawArrays(GL_TRIANGLE_FAN, 8, 12);`
3. Реализация — драйвер / Mesa.

Библиотеки вокруг API:

- `glfwCreateWindow` — где взять framebuffer;
- `glewInit` — где взять адрес `glDrawArrays`;
- `glm::vec3` — какие числа положить в VBO.

Mesa

Mesa — открытый проект, который предоставляет реализацию различных графических API (таких как OpenGL, OpenGL ES, Vulkan, OpenCL и др.).

Mesa поддерживает реализацию OpenGL для устройств, разработанных Intel, AMD, NVIDIA, Qualcomm и др., включая виртуальные графические процессоры VMware и VirGL. Также есть несколько программных рендереров: Softpipe (драйвер Gallium) и LLVMpipe (высокоскоростной растеризатор на основе LLVM/JIT).

2026-09-19

Построение изображений

└ Графический стек приложения

└ Mesa

Mesa

Mesa — открытый проект, который предоставляет реализацию различных графических API (таких как OpenGL, OpenGL ES, Vulkan, OpenCL и др.).

Mesa поддерживает реализацию OpenGL для устройств, разработанных Intel, AMD, NVIDIA, Qualcomm и др., включая виртуальные графические процессоры VMware и VirGL. Также есть несколько программных рендереров: Softpipe (драйвер Gallium) и LLVMpipe (высокоскоростной растеризатор на основе LLVM/JIT).

Проект начал разрабатывать Брайан Пол (Brian Paul) в 1993 году.

Основная цель заключалась в реализации API, совместимого с OpenGL, по мотивам IRIS GL.

Первая версия вышла в 1995 году с разрешения SGI. Название Mesa, по словам Пола, просто пришло в голову; SGI запретила использовать в имени слова «Open» и «GL».

<https://docs.mesa3d.org/history.html>

Распределение вычислений между CPU и GPU

CPU (Central Processing Unit) и GPU (Graphics Processing Unit)

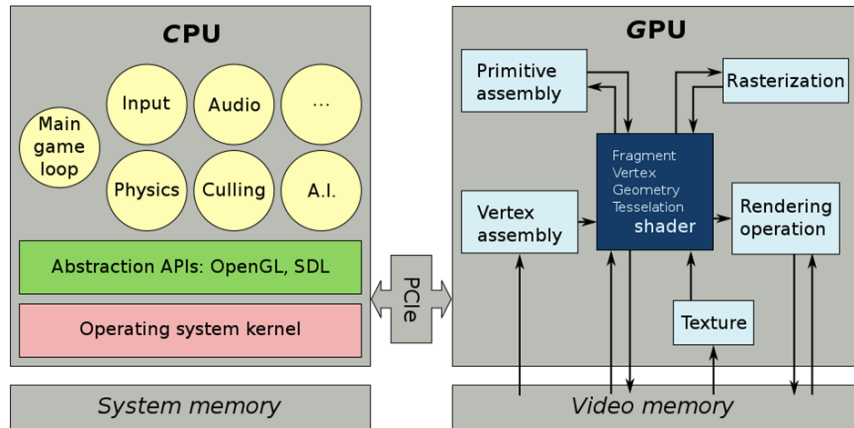


Рис. 4: Принципиальная схема распределения вычислений

2026-09-19

Построение изображений

└ Распределение вычислений

└ Распределение вычислений между CPU и GPU

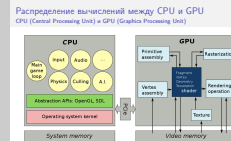


Рис. 4: Принципиальная схема распределения вычислений



Рис. 5: Ghost of Tsushima (2024, Призрак Цусимы)

Упрощённая схема графического конвейера

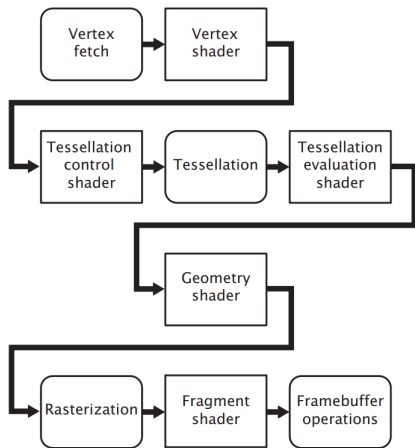


Рис. 6: Порядок вычисления шейдеров

Загрузка данных

Вершины (vertices)

Вершинный шейдер

Группа вершин (primitives/patches)

Шейдер управления тесселяцией

Тесселяция

Шейдер оценки тесселяции

Примитивы (primitives)

Геометрический шейдер

Примитивы (primitives)

Растрезация и интерполяция

Фрагменты (fragments)

Пиксельный (фрагментный) шейдер

Фрагменты (fragments)

Операции с буфером кадра

Пиксели (pixels)

2026-09-19

Построение изображений

└ Модель графического конвейера

└ Упрощённая схема графического конвейера



Рис. 7: DirectX 11 (2009): без тесселяции (слева) и с тесселяцией (справа)

Вершинный и пиксельный шейдеры:

DirectX 8.0, 2000; OpenGL 2.0, 2004

Геометрический шейдер:

DirectX 10, 2006; OpenGL 3.2, 2009

Шейдеры тесселяции:

DirectX 11, 2009; OpenGL 4.0, 2010

Схема графического конвейера

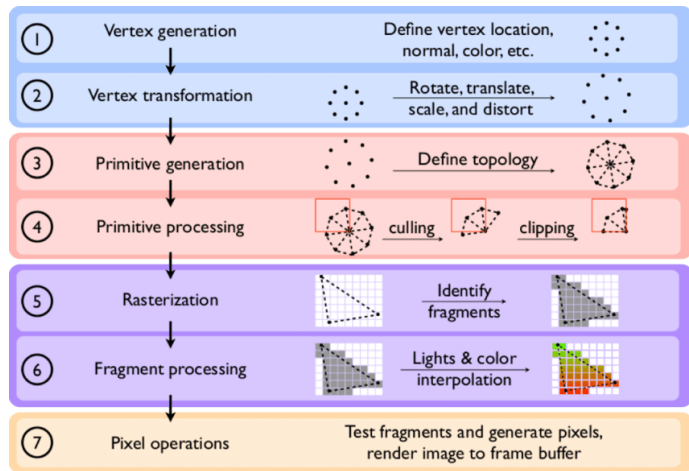


Рис. 8: Схема графического конвейера

2026-09-19

Построение изображений

└ Модель графического конвейера

└ Схема графического конвейера

Схема графического конвейера

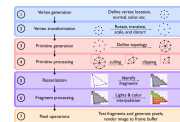


Рис. 8: Схема графического конвейера

Растеризация (Rasterization)

Растеризация — метод рендеринга, который преобразует 3D-сцену в 2D-изображение, отображаемое на экране. Этот метод долгое время является основным подходом в компьютерной графике для создания изображений в реальном времени и отличается высокой производительностью, так как оптимизирован для работы на GPU.

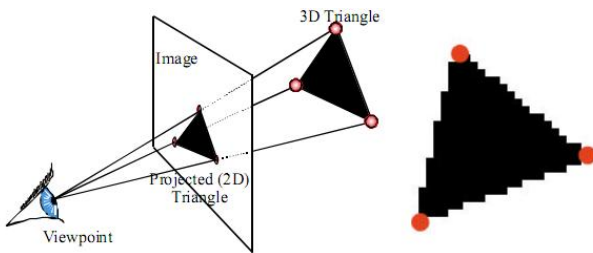


Рис. 9: Схема построения изображения с помощью растеризации

2026-09-19

Построение изображений

- Модель графического конвейера

- Растеризация (Rasterization)

Растеризация (Rasterization)

Растеризация — метод рендеринга, который преобразует 3D-сцену в 2D-изображение, отображаемое на экране. Этот метод долгое время является основным подходом в компьютерной графике для создания изображений в реальном времени и отличается высокой производительностью, так как оптимизирован для работы на GPU.

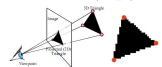


Рис. 9: Схема построения изображения с помощью растеризации

Растр (Raster, 1934) — сканирующее поле.

Недостаток растеризации заключается в том, что тени и отражения приходится добавлять отдельными методами.

Конвейер рисования в OpenGL

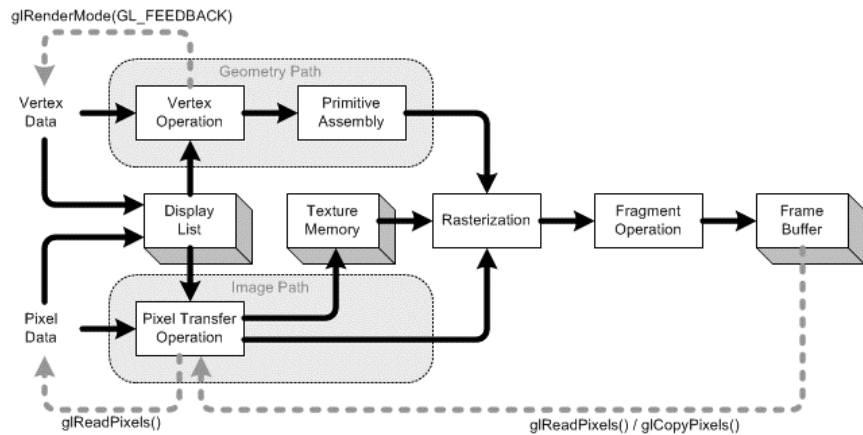


Рис. 10: Конвейер рисования OpenGL

2026-09-19

Построение изображений

└ Модель графического конвейера

└ Конвейер рисования в OpenGL

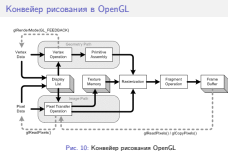


Рис. 10: Конвейер рисования OpenGL

Характеристики видеокарты

Графический процессор (видеочип)

Число транзисторов, млрд.

Техпроцесс, нм

Тактовая частота, ГГц

Количество шейдерных ядер (ALU, Arithmetic Logic Unit)

Тензорные ядра (Tensor Cores, NVIDIA)

RT-ядра (Ray-Tracing Cores, NVIDIA)

Блоки растровых операций (ROP)

Блоки наложения текстур (TMU)

Скорость заполнения (Fill Rate)

Пиксельная — ROPs × частота

Текстурная — TMUs $\times$ частота

Графическая память

Разрядность шины, бит

Тип микросхем (GDDR7 SDRAM)

Тактовая частота, ГГц

Объём, Гбайт

Другие атрибуты

Размеры

Тип охлаждения

Шина I/O (PCIe)

Мощность, Вт (Энерговыделение)

Производительность шейдерных ALU

(FP32/FP64/FP16)

Построение изображений

Характеристики видеокарты

└ Характеристики видеокарты

Описание самой мощной игровой видеокарты на 2026 г. NVIDIA

GeForce RTX 5090

<https://www.techpowerup.com/gpu-specs/geforce-rtx-5090.c4216>

С различными характеристиками можно ознакомиться:

https://www.thg.ru/graphic/graphic_card_faq_ii/index.html

Скорость заполнения (Fill Rate)

С какой скоростью графический процессор может выдавать пиксели. Выделяют два типа скорости заполнения: пиксельную (pixel fill rate, $PFR = ROP \cdot Hz$) и текстурную (texture fill rate, $TFR = TMU \cdot Hz$).

Исторически (середина 2000-х) ATI и nVidia считали текстурную скорость заполнения по-разному. nVidia умножала число пиксельных конвейеров на тактовую частоту, ATI — число текстурных блоков на тактовую частоту. Оба способа были корректны для тогдашних архитектур, где у nVidia был один текстурный блок на пиксельный конвейер.

Блоки наложения текстур (Texture Mapping Unit, TMU)

Текстуры следует выбрать и отфильтровать. Эта работа выполняется блоками наложения текстур, которые работают совместно с блоками пиксельных и вершинных шейдеров. Работа TMU заключается в применении текстурных операций над пикселями.

Блоки растровых операций (Raster Operations Pipeline, ROP)

Процессоры растровых операций отвечают за запись пиксельных данных в память. Скорость, с которой выполняется эта операция, является скоростью заполнения (fill rate).

Производительность (и число) ROP уже редко используется для оценки скорости видеокарты, т.к. перестала быть узким местом.

Характеристики видеокарты

Memory Hierarchy

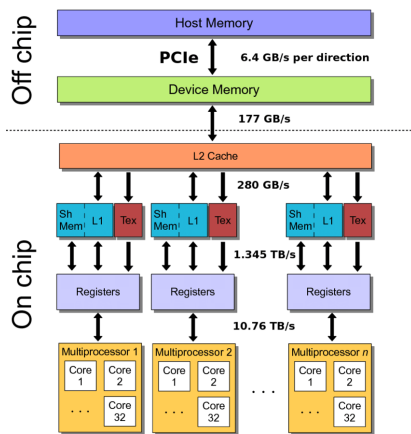


Рис. 11: Nvidia Fermi Architecture Memory

2026-09-19

Построение изображений

└─ Характеристики видеокарты

└─ Характеристики видеокарты

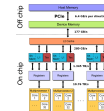


Рис. 11: Nvidia Fermi Architecture Memory